WORCESTER POLYTECHNIC INSTITUTE
ROBOTICS ENGINEERING PROGRAM

Lab 2 – Forward Kinematics

SUBMITTED BY
Sukriti Kushwaha
Istan Slamet
Tracy Yang

Date Submitted: September 14, 2023
Date Completed: September 14, 2023
Course Instructor: Dr. Agheli
Lab Section: RBE 3001 A'12

Abstract

This report overviews the experiments performed in Lab 2, which was used to learn how to implement forward kinematics on an OpenManipulator robotic arm. The experiments primarily included graphical and computational analyses of the arm's kinematics and joint positions while undergoing several different movements. All experiments were performed using MATLAB.

Introduction

This lab uses forward kinematics calculations to determine the position and orientation of the end effector with input joint values. Joint values corresponding to each servo motor are sent into DH parameters and inserted into a forward kinematics matrix of its respective joint. All of the matrices are then multiplied together to create the forward kinematics matrix of the entire arm, solving for the position and orientation of the end effector. With the position and orientation, we can create plots that mimic the robot's links and have it update live with the arm as well as update the vertices of each joint.

Methodology

Before beginning, we made sure to understand our arm and its axes as well as solve for the DH parameters. We did so to also have the names of joints, frames, and transformation matrices all be consistent.

Once solved, we coded 3 functions to calculate forward kinematics: dh2mat(), dh2fk(), and fk3001(). The first of the three functions takes in 4 joint positions in degrees, inputs it into the DH parameter table, and outputs a homogeneous transformation matrix. The second uses the previous function to solve for a composite transformation matrix with the input of n sets of joint positions. The final function takes in nx1 column vector of joint positions and returns a homogenous transformation matrix of the entire arm with respect to the base frame.

After we finished writing our functions, we tested them on the robot for Sign-off #1. To do this, we sent each function a series of arbitrary values. Using our own calculations, we determined whether or not each function was outputting the correct values. We tested dh2mat() by inputting the home position DH parameters for each link of the robot. We multiplied the results together to calculate the T_{Base}^{Tip} homogenous transformation matrix. We compared this with our calculations and determined them to be correct. We tested fk3001() similarly, by inputting the home position values, and found the result to be the correct homogeneous transformation matrix. We knew that dh2fk() worked properly because fk3001() implements dh2fk().

We then created functions measured_cp(), setpoint_cp(), and goal_cp() which take in the data from the similarly named lab 1 functions and returns a homogenous transformation matrix of the current joint positions, the current joint set point positions, and the end of motion joint set point positions, respectively.

We tested these functions for number 3 (Sign-off #2) by interpolating the arm to two arbitrary positions starting at the home position. In a section, we called measured_cp() and setpoint_cp() before interpolating and called goal_cp() after interpolating. This resulted in three different homogenous transformation matrices which corresponded to the movements' current positions, current set positions, and final position. We compared this to our own calculations and determined them to be correct.

Next, for number 4, we commanded the robot to move away from the home position to an arbitrary point and back again five times. We recorded the homogenous transformation matrices for each time the robot returns. To do this, we added each matrix to a cell array, celery1(). We ran a for loop to repeat the movements while also using measured_cp() to keep track of the joint values while in movement. Next, we 3D-plotted each of the tip positions with respect to the base frame using scatter3(). We also used scatter3() to plot the ideal position for the movement as well as the average of positions for the movements within the same plot. We also found the root mean square error values of the tip position for each run.

For number 5, we needed to create the function plot_arm(), which would create a 3D stick model of the arm showing all frames, joints, and links. We started by creating a new class named Model and created the plot_arm() function within it. The function takes in a 1x4 array q which corresponds to the robot's joint configuration. We started writing the function by creating a matrix L to store the link lengths. We then created the DH tables for each of the joints and end-effector based on the function's input. Then we multiplied each of the DH tables. Then we created the homogeneous transformation matrices for each table by using dh2fk() and isolated the translation matrices into a separate set of variables. We then used plot3() to plot each translation matrix in 3D space. To create the joint axes, we used quiver3 at each joint position and used length 50 for each unit vector. We tested the function with three arbitrary robot positions.

For number 6, we needed to create a live plot of the robot in a separate script called "lab2_live_plot.m". This live plot would continuously plot the positions of the robot arm as it moved. First, we created a matrix m to hold joint angles for each separate movement. Then we used a for loop to iterate through m, using interpolate_jp() to move the robot to each position. In a while loop, we used a timer to continuously call plot_arm(), inputting setpoint_js(). This continuous call would make plot_arm() repeatedly plot the set positions of the robot for each timer tick. We demonstrated the results for Sign-off #3.

For number 7, we needed to create a script to move the robot to the three vertices of a triangle in the x-z plane while continuously recording the joint angles and end-effector's position into a .csv file. To store these values, we created three separate matrices: time, for time values; joints, for position values; and eePos, for the end-effector positions. In a while loop, we continuously concatenated these matrices and added the respective values while the robot was in motion. We then combined all of these matrices into one matrix, jointsEEpos, which we sent to the .csv file 'trajectory.csv' and used for plotting. Using, plot(), we created several plots from this matrix, including Joint Angles Over Time, XZ End Effector Positions Over Time, XZ Trajectory, and XY Trajectory. This also completed Sign-off #4.

Results

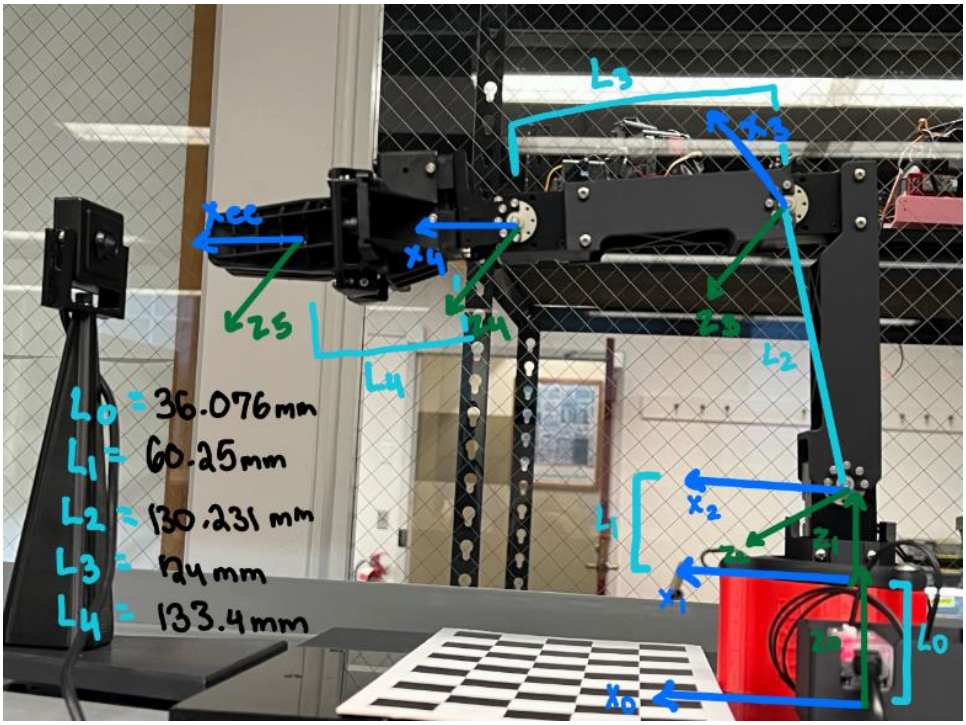


Figure 1: A picture of the arm depicting all joints and joint directions

	θ	d	a	α
F_0^1	0	L_0	0	0
F_1^2	0*	L_1	0	$-\pi/2$
F_2^3	$-\tan^{-1}\left(\frac{128}{24}\right)+0^*$	0	L_2	0
F_3^4	$\tan^{-1}\left(\frac{128}{24}\right)+0^*$	0	L_3	0
F_4^{ee}	0*	0	L_4	0

Figure 2: A table of DH parameters

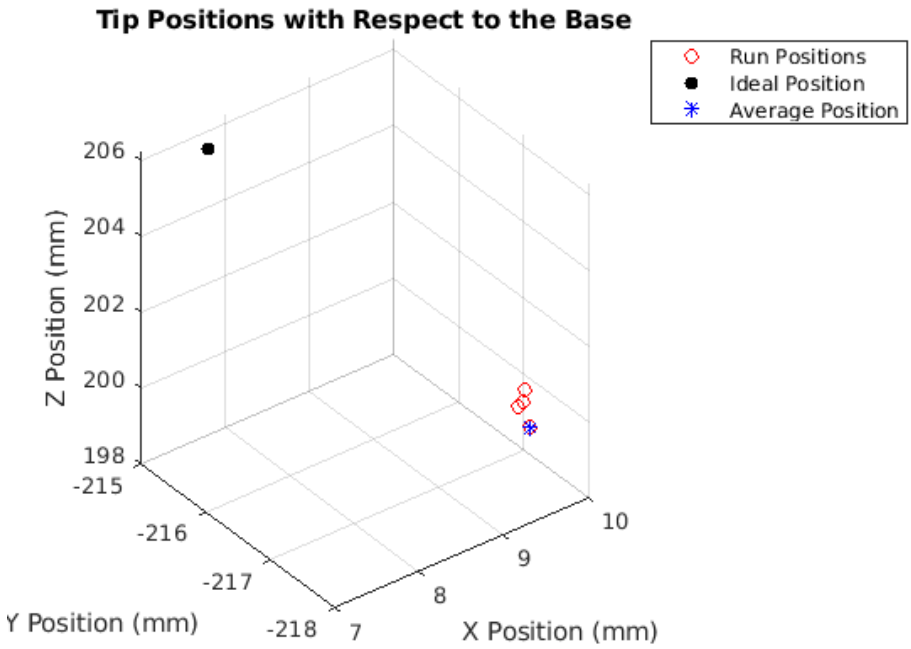


Figure 3: A 3D scatter plot of 5 tip positions with respect to the base frame

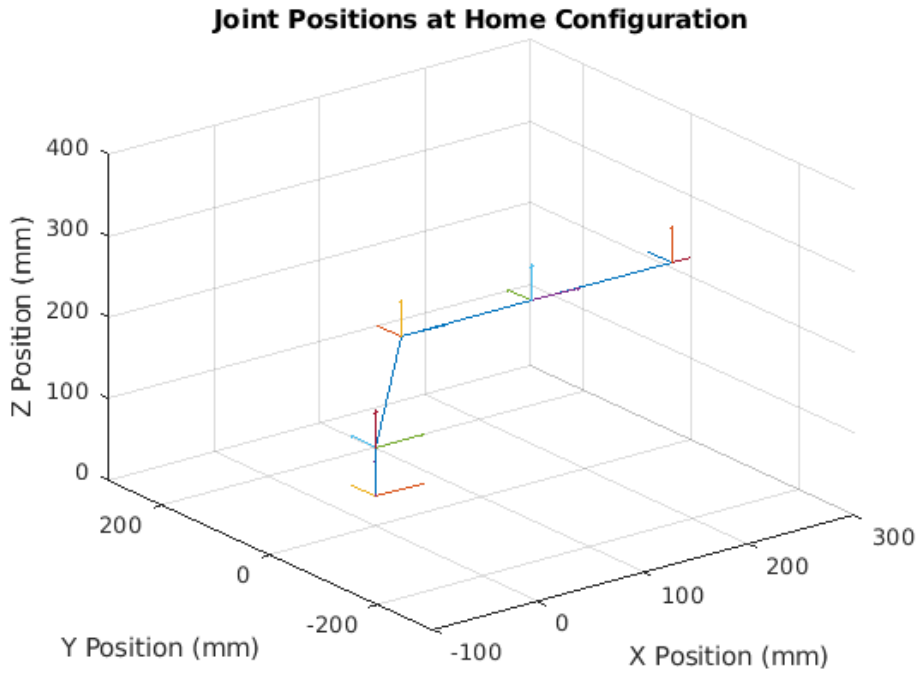


Figure 4: A stick model of joints at home configuration

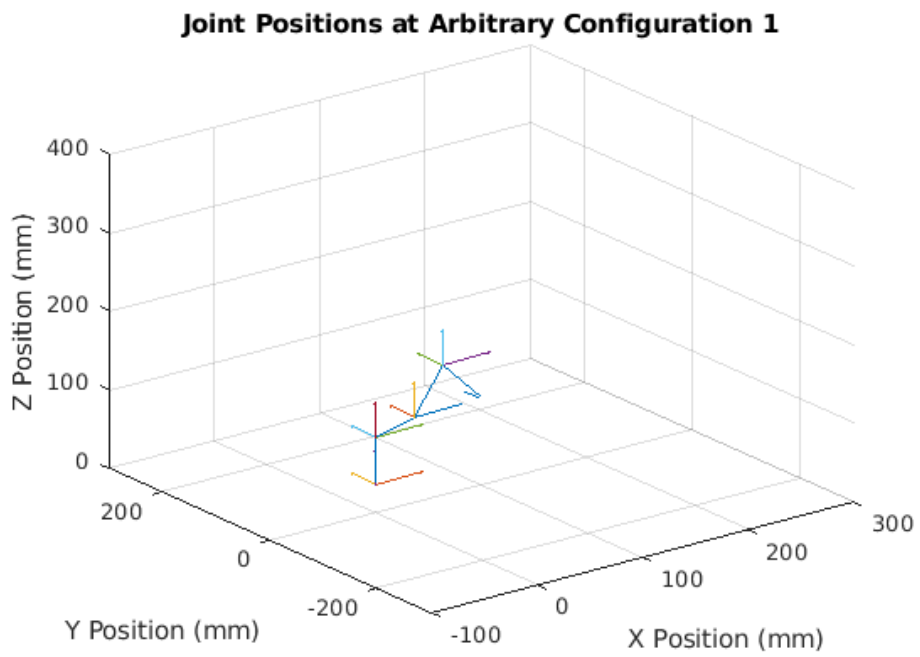


Figure 5: A stick model of joints at arbitrary configuration 1

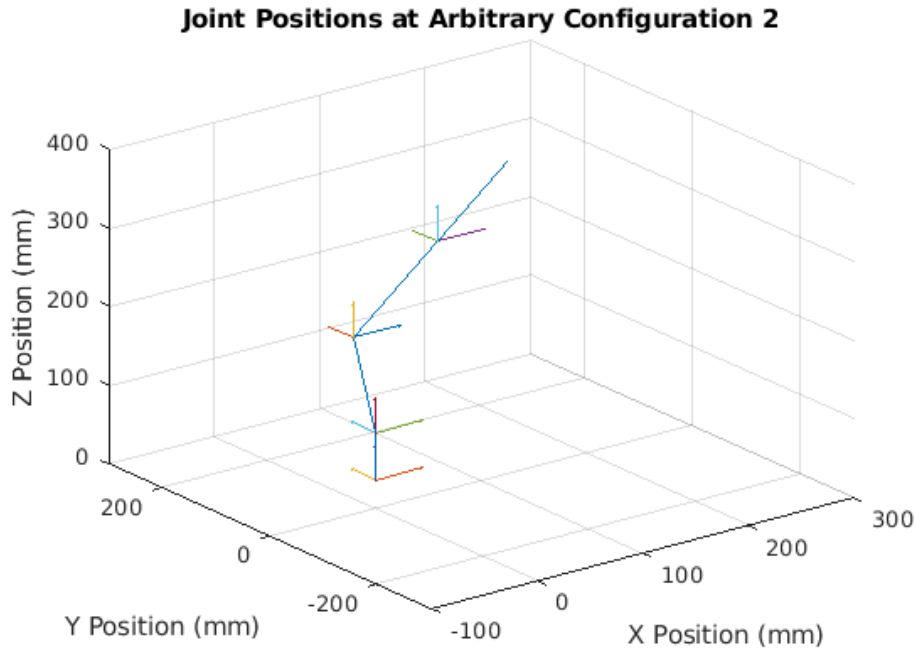


Figure 6: A stick model of joints at arbitrary configuration 2

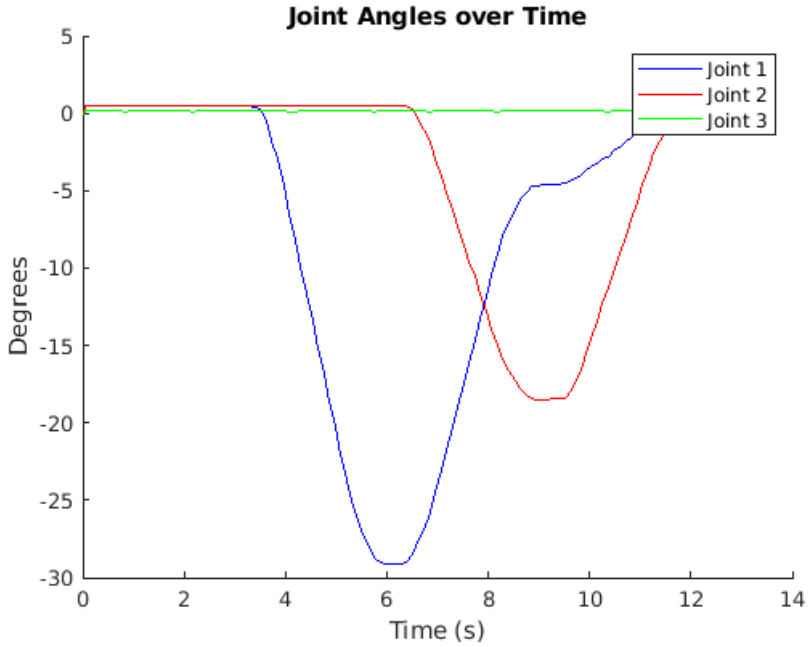


Figure 7: A plot representing joint angles of triangle trajectory with respect to time

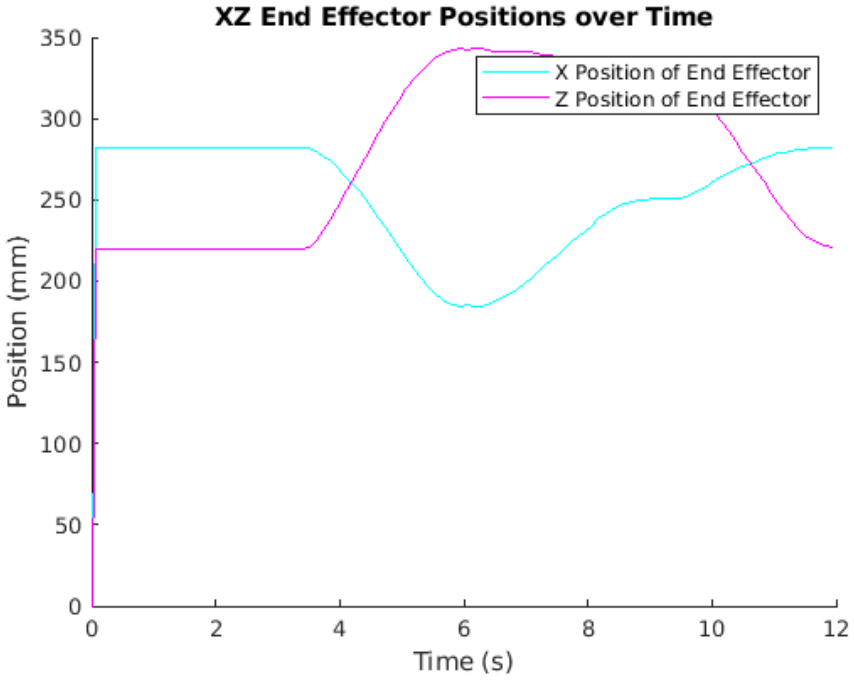


Figure 8: A plot representing end effector positions of x and z versus time

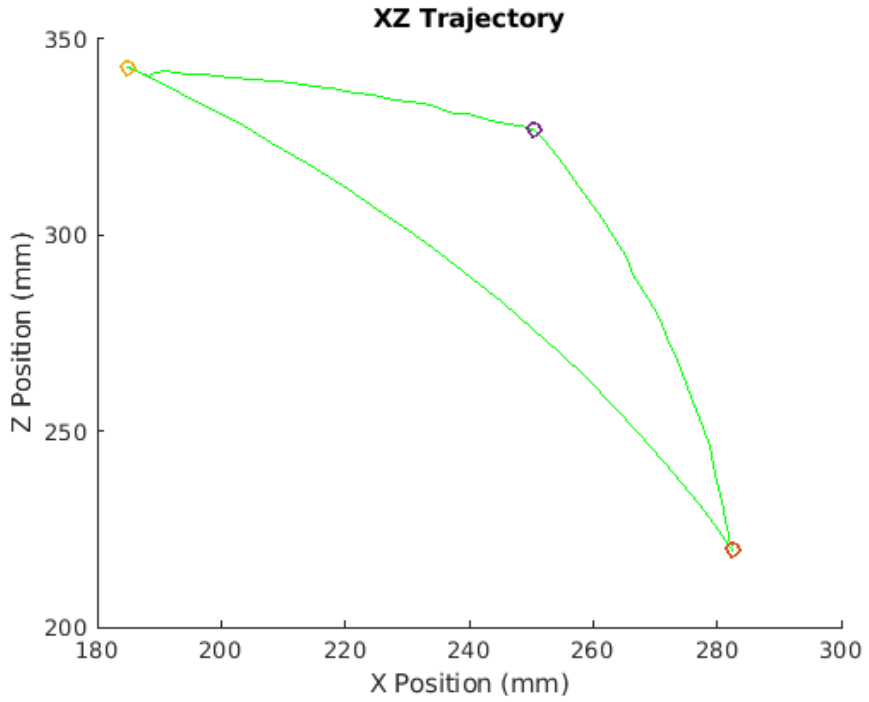


Figure 9: The 2D path representing the trajectory of arm in the X-Z plane overlayed with triangle vertices

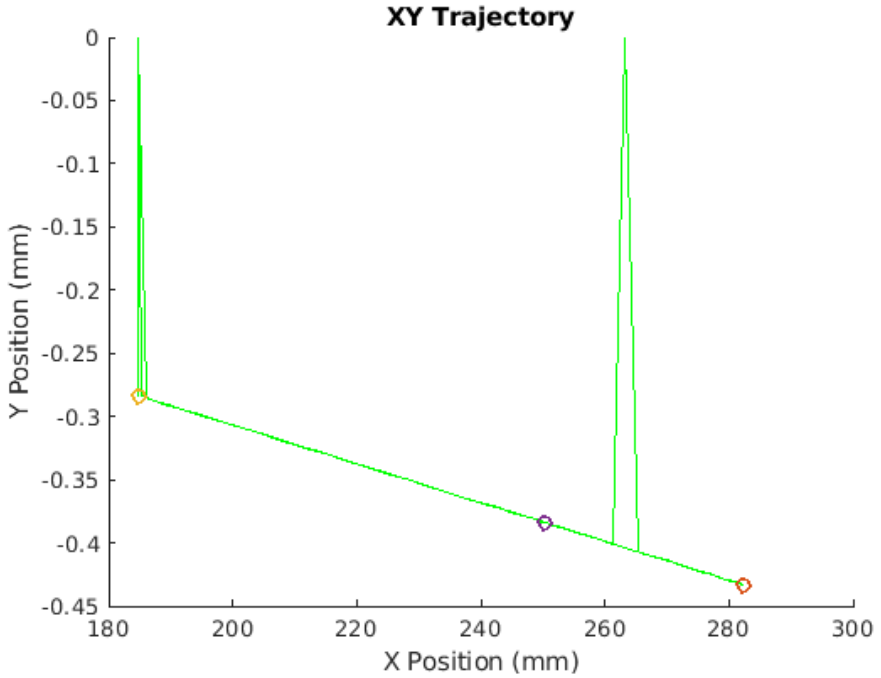


Figure 10: The 2D path representing the trajectory of arm in the X-Y plane overlayed with triangle vertices

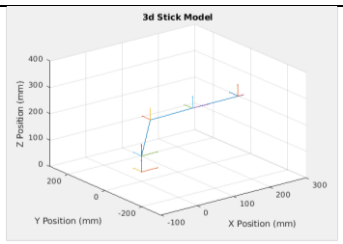
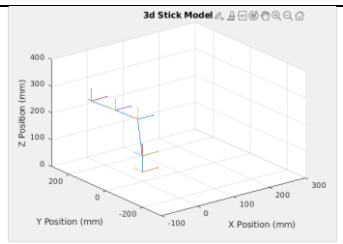
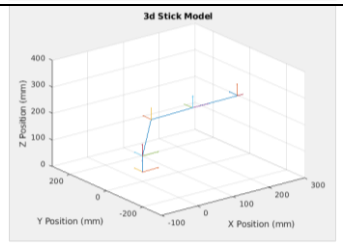
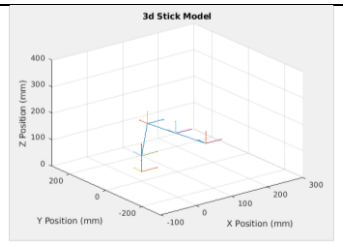
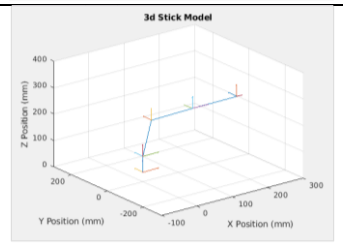
1		
2		
3		
4		
5		

Figure 11: The plots and pictures of the 5 positions used in the positions being as follows: [90 0 0 0], [-90 0 0 0], [0 0 0 0], [90 0 0 0], and [0 0 0 0]

	Position(mm)		
Configuration #	X	Y	Z
1	9.68	-217.44	200.46
2	9.35	-217.61	200.13
3	9.35	-217.46	200.46
4	9.68	-217.44	200.46
5	9.34	-217.37	199.88
Average	**9.48**	**-217.46**	**200.278**
Root Mean Square Error	**3.42**		

Table 1: A table depicting the average tip position and root mean square error of the five joint configurations

Frame	Homogenous Transformation Matrix			
0 to 1	1	0	0	0
	0	1	0	0
	0	0	1	$\frac{9019}{250}$
	0	0	0	1
1 to 2	1	0	0	0
	0	0	1	0
	0	-1	0	$\frac{9019}{250}$
	0	0	0	1
2 to 3	$\frac{3 * 265^{\frac{1}{2}}}{265}$	$\frac{16 * 265^{\frac{1}{2}}}{265}$	0	$\frac{6755422017634643}{281474976710656}$
	$-\frac{16 * 265^{\frac{1}{2}}}{265}$	$\frac{3 * 265^{\frac{1}{2}}}{265}$	0	$-\frac{2251807339211549}{17592186044416}$
	0	0	1	0
	0	0	0	1
3 to 4	$\frac{3 * 265^{\frac{1}{2}}}{265}$	$-\frac{16 * 265^{\frac{1}{2}}}{265}$	0	$\frac{804025472225023}{35184372088832}$
	$\frac{16 * 265^{\frac{1}{2}}}{265}$	$\frac{3 * 265^{\frac{1}{2}}}{265}$	0	$\frac{8576271703733583}{70368744177664}$
	0	0	1	0
	0	0	0	1
4 to 5	1	0	0	$\frac{667}{5}$
	0	1	0	0
	0	0	1	0
	0	0	0	1
Composite	1	0	0	$\frac{1237610640983711}{4398046511104}$
	0	0	1	0
	0	-1	0	$\frac{7892784504251929}{35184372088832}$
	0	0	0	1

Table 2: The intermediate homogenous matrices of the function dh2mat() at the home configuration

Configuration	Homogenous Transformation Matrix			
Home	1	0	0	$\frac{1237610640983711}{4398046511104}$
	0	0	1	0
	0	-1	0	$\frac{7892784504251929}{35184372088832}$
	0	0	0	1
Arbitrary 1	$\frac{-6256334945874825}{36028797018963968}$	0	$\frac{-8870359658994711}{9007199254740992}$	$\frac{-3973483984197353}{70368744177664}$
	$\frac{-4435179829497355}{4503599627370496}$	$\frac{-1}{18014398509481984}$	$\frac{6256334945874825}{36028797018963968}$	$\frac{-1408421717507537}{4398046511104}$
	$\frac{-1}{9007199254740992}$	-1	0	$\frac{566571599302917}{2199023255552}$
	0	0	0	1
Arbitrary 2	$\frac{5789716078925341}{9007199254740992}$	$\frac{3449957468579869}{4503599627370496}$	0	$\frac{634319272308419}{4398046511104}$
	0	0	1	0
	$\frac{3449957468579869}{4503599627370496}$	$\frac{-5789716078925341}{9007199254740992}$	0	$\frac{1855955431726943}{4398046511104}$
	0	0	0	1
Arbitrary 3	$\frac{3449957468579869}{4503599627370496}$	$\frac{1447429019731335}{2251799813685248}$	0	$\frac{-3973483984197353}{70368744177664}$
	0	0	1	0
	$\frac{1447429019731335}{2251799813685248}$	$\frac{-3449957468579869}{4503599627370496}$	0	$\frac{7165064489375197}{17592186044416}$
	0	0	0	1

Table 3: The homogenous matrices of the function fk3001() at the home position, [80; -70; -60; -50], [0; -20; -30; 0], and [0; -10; -40; 10]

Configuration	Homogenous Transformation Matrix			
[90 0 0 0]	0.00306570103943139	-0.00011762583159297	-0.999995293809576	0.869458880093251
	0.999260044556410	-0.0383399399376454	0.00306795676296598	283.398644579868
	-0.0383401203735527	-0.999264747286595	0	214.985005048671
	0	0	0	1
[-90 0 0 0]	0.00153285232319049	-5.88129849928027e-05	0.999998823451702	0.435030105742747
	-0.999263571603357	0.0383400752645493	0.00153398018628477	-283.595314853734
	-0.0383401203735527	-0.999264747286595	0	214.944731528193
	0	0	0	1
[0 0 0 0]	0.999260044556410	-0.0383399399376453	0.00306795676296598	283.594313862586
	-0.00306570103943140	0.000117625831592971	0.999995293809576	-0.870059187817634
	-0.0383401203735527	-0.999264747286595	0	214.944731528193
	0	0	0	1
[90 0 0 0]	0.00306570103943140	-0.00011762583159297	-0.999995293809576	0.870059187817634
	0.999260044556410	-0.0383399399376453	0.00306795676296598	283.594313862586
	-0.0383401203735527	-0.999264747286595	0	214.944731528193
	0	0	0	1
[0 0 0 0]	0.999260044556410	-0.0383399399376453	0.00306795676296598	283.594313862586
	-0.00306570103943140	0.000117625831592971	0.999995293809576	-0.870059187817634
	-0.0383401203735527	-0.999264747286595	0	214.944731528193
	0	0	0	1

Table 4: The homogenous transformation matrices of the 5 set positions.

Discussion

Figure 1 displays the robotic arm's joint axes and link lengths. Identifying these was a key step for the creation of Figure 2, which is a DH table for each of the links of the robot. It can be seen from the table that each of the joints are revolute, with only one joint at a separate alpha angle than the rest. The lengths of the links can also be found in Figure 1.

For number 2, we tested the dh2mat() and fk3001() functions and plotted the arm positions in Figures 3, 4, and 5. From these graphs, we could infer that the functions were working correctly as the plotted values matched our predictions.

For number 4, when the robot arm moved to an arbitrary position and back home, we noticed that there was very little discrepancy between the tip positions for each round we went back and forth. In Figure 3, the points on the scatter plot almost completely overlay each other. Because of this, we can infer that the error is random as there is no pattern. The average error from the goal position is about 7mm, which is small enough to be deemed not significant. The root mean square error is also low at 3.42 which adds to

this point. We believe that the cause is loose screws between joints in the robotic arm causing the robot to hang slightly lower than expected.

For number 5, when plotting the zero and arbitrary configurations of the joints we found that the stick model mimicked the robot's orientation, indicating that we had accurately created the 3D stick model plot. This is shown in figure 11, where all of the pictures mirror the live plot screenshots.

For number 6, we attempted to have the path be a triangle on the XZ plane. Shown in Figure 9, the triangle did not have straight lines but had 3 points connected by curves. If we used inverse kinematics to have the end effector go to the position rather than have the joint positions determine the end effector's position, the trajectory path could look more triangle-like. However, using forward kinematics, we were still able to command the arm back to the original position and close the triangle.

Conclusion:

Overall, this lab taught us how to calculate the position of the end effector of our robot with respect to the base frame using forward kinematics. More specifically, we utilized DH parameters and homogenous transformation matrices to calculate the end effector positions given the initial joint angles as input. We visualized the position of the end effector by plotting the tip with respect to the base frame and compared its accuracy between intervals of the same configuration. Additionally, we utilized the plot3() and quiver() functions to generate a 3d stick model of our robot and update it in live time as our robot moves. We finished the lab by plotting the trajectory of our robot as it moved through the vertices of a triangle, showing that forward kinematics is not suitable for moving along a specific path. We believe inverse kinematics would be better suited for such a task!

Appendix A: Authorship

Section	Author
Introduction	Tracy Yang
Methodology	Istan Slamet
Results	Suki Kushwaha and Tracy Yang
Discussion	Everyone 😊
Conclusion	Suki Kushwaha